

# HIVE: A JaCaMo-based Multi-Agent System for the Multi-Agent Programming Contest 2022

PCS 5703 — Multi-Agent Systems  
Escola Politécnica, University of São Paulo  
1<sup>st</sup> Term, 2026

## Abstract

Multi-Agent Systems (MAS) based on the BDI (*Belief-Desire-Intention*) architecture [3, 4] represent a promising approach to distributed coordination problems, yet face significant practical challenges when applied to competitive scenarios with strict temporal constraints. This paper presents **HIVE** (*Hierarchical Intelligent Virtual Ensemble*), a MAS developed on the JaCaMo platform [6] for the *Agents Assemble* scenario of the 2022 *Multi-Agent Programming Contest* (MAPC) [14]. The system employs **20 BDI agents** programmed in AgentSpeak(L)/Jason [8] — as mandated by the official competition scenario —, organized into 4 autonomous squads via MOISE+ [9], with CArtAgO artifacts [11] for shared map management, task board, and squad coordination. Task allocation uses a distributed auction inspired by the *Contract Net Protocol* [22], complemented by a universal soloist pool that maximizes agent utilization. HIVE adapts to the official scenario through **reactive role adoption** (the `default` role cannot collect or submit until `adopt(worker)` on a *role zone*) and **relative positioning** via dead-reckoning (`absolutePosition: false`). A scalability study in a development scenario (6, 10, 15, and 20 agents) demonstrates scores of 60–100 points (mean 77.1) at the 15-agent configuration on the reduced grid, positioning HIVE in the mid-to-competitive range of MAPC 2022 teams.

**Keywords:** Multi-Agent Systems, BDI, JaCaMo, Jason, MOISE+, CArtAgO, MAPC, Contract Net Protocol.

## 1 Introduction

### 1.1 The Problem

The **Multi-Agent Programming Contest** (MAPC) is an annual competition organized since 2005 by Clausthal University of Technology, aiming to stimulate research in the development and programming of multi-agent systems by identifying key problems, collecting suitable benchmarks, and gathering test cases that require coordinated action [16, 24]. In its 2022 edition, the **Agents Assemble III** scenario [14, 23] challenges two teams of **20 agents** to operate simultaneously on a toroidal **70×70** grid, partially observable, with the following tasks:

- **Explore** the map with limited vision (radius of 5 cells);
- **Collect blocks** from dispensers scattered across the environment (actions `request` + `attach`);
- **Assemble structures** by connecting blocks between adjacent agents (action `connect`);
- **Submit tasks** in goal zones according to server-defined patterns (action `submit`);
- **Respect dynamic norms** that limit block transport.

The complexity stems from the combination of partial observability, dynamicity (tasks appear and expire, goal zones move, norms change), temporal constraints (4–10 second timeout per step), **relative positioning** (in the official scenario, `absolutePosition: false` — the agent perceives only relative coordinates), the need for **role adoption** on *role zones* to enable collection/submission actions, and coordination among multiple agents for multi-block structures.

## 1.2 Software Used

HIVE is implemented on the **JaCaMo** platform [6, 25], which integrates:

- **Jason** [8]: an AgentSpeak(L) interpreter for BDI agents;
- **MOISE+** [9, 10]: a normative organizational model;
- **CArtAgO** [11]: an artifact infrastructure for the computational environment.

The simulation server is **MASSim 2022-1.1** [23], communicating via the **EIS** (*Environment Interface Standard*) middleware. The build system uses **Gradle** with **Java 21**.

## 1.3 Hardware Used

| Component         | Specification                                        |
|-------------------|------------------------------------------------------|
| Processor         | Apple Silicon (ARM64)                                |
| RAM               | 16 GB                                                |
| JVM heap (agents) | 3 GB ( <code>-Xmx3g</code> , <code>-Xms512m</code> ) |
| Operating system  | macOS (Darwin)                                       |
| Agent timeout     | 8,000 ms (server and EIS)                            |

## 1.4 Contributions

The main contributions of this work are: (i) a complete MAOP architecture for MAPC 2022 with 20 agents, demonstrating the feasibility of JaCaMo in competitive scenarios [7]; (ii) a MOISE+ organization with a universal soloist pool [10]; (iii) mechanisms for adapting to the official scenario — reactive role adoption and relative positioning via dead-reckoning; (iv) an experimental scalability analysis revealing a serialization bottleneck in CArtAgO [12]; and (v) scores of 60–100 points (mean 77), comparable to declarative implementations such as the LI(A)RA team [19].

# 2 Analysis and Specification of the MAS

## 2.1 Development Method

Development followed the **Multi-Agent Oriented Programming** (MAOP) paradigm [6, 7], which structures MAS construction along three orthogonal dimensions: (i) **agents** — individual deliberative logic; (ii) **environment** — shared resources and tools; and (iii) **organization** — social constraints on collective behavior. The choice of JaCaMo is justified by adherence to the PCS 5703 course requirements and by the platform’s maturity in MAPC competitions [20, 21].

The development approach was **iterative and experimental**: each optimization was implemented, tested over 3+ independent simulations, and evaluated comparatively before being kept or reverted.

## 2.2 MAS Requirements

The functional requirements were derived directly from the Agents Assemble scenario [23]:

| Requirement | Description                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------|
| FR01        | Explore the toroidal map with partial vision (radius 5)                                                                        |
| FR02        | Locate dispensers and goal zones                                                                                               |
| FR03        | Collect the correct block type at dispensers                                                                                   |
| FR04        | Navigate with attached blocks to goal zones                                                                                    |
| FR05        | Submit single-block tasks in goal zones                                                                                        |
| FR06        | Coordinate assembly of multi-block structures (connect)                                                                        |
| FR07        | Respect dynamic norms (e.g., block carry limit)                                                                                |
| FR08        | Allocate tasks among agents efficiently                                                                                        |
| FR09        | Recover from blocking situations ( <i>stuck detection</i> )                                                                    |
| FR10        | Adopt the <b>worker</b> role on <i>role zones</i> (official scenario; <b>default</b> role lacks collection/submission actions) |
| FR11        | Operate with relative positioning ( <b>absolutePosition: false</b> ) via dead-reckoning                                        |

Non-functional requirements:

| Requirement | Description                                  |
|-------------|----------------------------------------------|
| NFR01       | Respond within the 8-second per-step timeout |
| NFR02       | Scale to 20 simultaneous agents              |
| NFR03       | Maintain consistency of the shared map       |

## 2.3 Agent Specification

The 20 HIVE agents follow the **BDI** (*Belief-Desire-Intention*) architecture [3, 4], grounded in Bratman’s theory of practical reasoning [3]: intentions are commitments that persist as admissibility filters, reducing the deliberation space — an essential property given the 8-second per-step timeout. The computational formalization by Rao and Georgeff [4, 5] translates this model into three mental attitudes:

- **Beliefs:** `my_pos(X,Y)`, `known_task(Name, Deadline, Reward, Size)`, `thing(X, Y, Type, Details)`;
- **Desires:** trigger events `!explore`, `!collect_block`, `!go_to(X,Y)`, `!submit_task`;
- **Intentions:** selected plans in execution — stacks of committed actions.

The agents are instantiated in 4 specialized roles (totaling 20 agents):

| Role                | Qty | Responsibility                                      |
|---------------------|-----|-----------------------------------------------------|
| <b>Squad Leader</b> | 4   | Task scan, auction, delegation to soloists          |
| <b>Collector</b>    | 8   | Block collection, navigation to meeting points      |
| <b>Assembler</b>    | 4   | <i>connect</i> protocol for multi-block, submission |
| <b>Sentinel</b>     | 4   | Autonomous soloist — independent collection         |

All agents share the common modules — `perception.asl`, `role_adoption.asl`, `connect_protocol.asl`, `collection.asl`, `navigation.asl`, `communication.asl`, `map_merge.asl`, among others — differing only in role-specific plans. This **compositional inheritance** ensures that any free agent can act as a soloist.

## 2.4 Organization Specification (MOISE+)

The organization is specified in MOISE+ [9, 10] across the model’s three dimensions:

**Structural Specification** — defines roles, groups, and relations:

- 4 roles: `squad_leader`, `collector`, `assembler`, `sentinel` (all extending `soc`);
- 1 group `hive_team` with a **flat structure** (cardinalities: 3–4 leaders, 6–8 collectors, 3–4 assemblers, 1–4 sentinels);
- Authority relations (`squad_leader` → `collector/assembler`) and communication (`collector` → `assembler`) intra-group;
- The **squad** concept is not encoded in the MOISE+ hierarchy but realized at runtime by the `SquadCoordinator` artifact (design decision U1, which simplifies instantiation and makes it robust).

**Functional Specification** — defines 3 schemes and their missions:

- `exploration_scheme` (mission `m_scout`): locate dispensers, goal zones, and *role zones*;
- `task_execution_scheme` (missions `m_collect`, `m_assemble`, `m_submit`): `collect` → `assemble` → `submit`;
- `defense_scheme` (mission `m_guard`): guard goal zones and clear threats.

**Deontic Specification** — 5 obligation norms bind roles to missions: `n_scout` (`squad_leader` → `m_scout`), `n_collect` (`collector` → `m_collect`), `n_assemble` and `n_submit` (`assembler`), `n_guard` (`sentinel` → `m_guard`).

The organization divides the 20 agents into 4 **autonomous squads** (1L + 2C + 1A each) plus a **pool of 4 sentinels**, inspired by decentralized military organizations where each squad has local tactical autonomy [9].

| Unit                   | Composition                                      |
|------------------------|--------------------------------------------------|
| Squad 1                | 1 Leader + 2 Collectors + 1 Assembler            |
| Squad 2                | 1 Leader + 2 Collectors + 1 Assembler            |
| Squad 3                | 1 Leader + 2 Collectors + 1 Assembler            |
| Squad 4                | 1 Leader + 2 Collectors + 1 Assembler            |
| Universal Soloist Pool | 4 Sentinels (receive free agents from any squad) |

Table 1: HIVE organization in MOISE+: 4 squads and the universal soloist pool.

## 2.5 Interaction Specification

Coordination among agents uses two complementary mechanisms:

**Distributed auction** (adaptation of the Contract Net Protocol [22]): when the server announces a task, each of the 4 leaders computes a bid based on reward and toroidal Manhattan distance to the nearest dispenser. The first leader to invoke `resolve_auction` on the `TaskBoard` artifact obtains the result — the squad with the highest score wins. The winning leader queries the `SquadCoordinator` to find the nearest free soloist and delegates via an ACL message.

**Self-assignment:** idle agents (with no delegated task) self-assign an available task from the belief base every 5 steps, maximizing utilization without coordination overhead.

**Computational stigmergy** [13]: the `SharedMap` artifact acts as a medium of indirect coordination — when an agent marks an obstacle or a visited cell, all other agents focusing the artifact perceive the change via observable properties, without direct point-to-point communication.

## 3 Architecture and Design of the MAS

### 3.1 Architecture Overview

HIVE operates as a client of the MASSim server, communicating via the EIS middleware. The architecture follows JaCaMo’s tripartite model [6]:

| Layer                           | Content                                               | Connection               |
|---------------------------------|-------------------------------------------------------|--------------------------|
| MASSim Server                   | Percepts, Tasks, Norms, Score                         | ↓ JSON/TCP               |
| EIS Middleware                  | Translates MASSim percepts → Jason beliefs            | ↓ beliefs                |
| Agent Dimension (Jason)         | 20 BDI agents (4 squads: L+2C+A) + 4 sentinels        | ↓ acts/observes          |
| Environment Dimension (CArtAgO) | SharedMap, TaskBoard, SquadCoordinator, HiveDashboard | —                        |
| Organization Dimension (MOISE+) | Roles, Groups, Schemes, Norms                         | regulates the Agent dim. |

Table 2: HIVE’s tripartite architecture over JaCaMo (top-down flow).

### 3.2 Internal Agent Architecture

Each agent implements a **layered hybrid architecture** combining BDI deliberation with a reactive hierarchy inspired by the subsumption architecture [1]:

| Priority | Layer       | Behavior                      | Module                            |
|----------|-------------|-------------------------------|-----------------------------------|
| 0        | Survival    | Deactivation, critical energy | <code>connect_protocol.asl</code> |
| 1        | Norms       | Detach excess blocks          | <code>connect_protocol.asl</code> |
| 2        | Submission  | Submit in goal zone           | <code>connect_protocol.asl</code> |
| 3        | Connection  | <i>connect</i> protocol       | <code>connect_protocol.asl</code> |
| 4        | Collection  | Request + attach at dispenser | <code>collection.asl</code>       |
| 5        | Navigation  | A* or greedy to destination   | <code>navigation.asl</code>       |
| 6        | Exploration | Unvisited frontier            | <code>navigation.asl</code>       |

The hierarchy is implemented by the **plan order** in the `.asl` files: Jason [8] selects the first applicable plan whose context guard is satisfied, ensuring that high-priority behaviors (P0) prevail over low-priority ones (P6), as shown in the table above.

### 3.3 Artifact Design

| Artifact                | Responsibility                                                                                                                                                                                                                                 | Main Operations                                                                                                                                                                                                                                        |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SharedMap</b>        | Parameterizable toroidal map (40×40 dev / 70×70 official), A*, frontiers, obstacles. In relative mode ( <code>absolutePosition: false</code> ), it becomes a per-agent instance ( <code>map_&lt;name&gt;</code> ) integrated by dead-reckoning | <code>mark_visited</code> , <code>compute_next_move</code> ,<br><code>get_nearest_frontier</code> ,<br><code>get_nearest_dispenser</code> ,<br><code>get_nearest_goal_zone</code> ,<br><code>get_nearest_role_zone</code> , <code>mark_obstacle</code> |
| <b>TaskBoard</b>        | Task registry and distributed auction                                                                                                                                                                                                          | <code>register_task</code> , <code>place_bid</code> ,<br><code>resolve_auction</code> , <code>is_task_assigned</code> ,<br><code>remove_expired</code>                                                                                                 |
| <b>SquadCoordinator</b> | Soloist pool and squad composition                                                                                                                                                                                                             | <code>find_free_soloist</code> ,<br><code>claim_task_soloist</code> , <code>mark_busy</code> ,<br><code>mark_free</code> , <code>release_agent</code>                                                                                                  |
| <b>HiveDashboard</b>    | Metrics and WebSocket broadcast                                                                                                                                                                                                                | <code>set_step</code> , <code>set_score</code> , <code>broadcast</code>                                                                                                                                                                                |

The artifacts follow the **A&A** meta-model [13]: they are passive entities that encapsulate functionality and mediate indirect coordination among agents. All operations on the same artifact are **serialized** by the CArtAgO runtime [12], guaranteeing sequential consistency.

### 3.4 Source-Code ↔ Architecture Mapping

| Component                      | File(s)                                                                    | Dimension    |
|--------------------------------|----------------------------------------------------------------------------|--------------|
| Perception and beliefs         | src/agt/common/perception.asl                                              | Agent        |
| Role adoption (official)       | src/agt/common/role_adoption.asl                                           | Agent        |
| Submission, norms, and connect | src/agt/common/connect_protocol.asl                                        | Agent        |
| Block collection               | src/agt/common/collection.asl                                              | Agent        |
| Navigation and exploration     | src/agt/common/navigation.asl                                              | Agent        |
| Communication and map merge    | communication.asl, map_merge.asl                                           | Agent        |
| Specialized roles              | squad_leader.asl, collector.asl, assembler.asl, sentinel.asl (in src/agt/) | Agent        |
| Shared map                     | src/env/env/SharedMap.java                                                 | Environment  |
| Task board                     | src/env/env/TaskBoard.java                                                 | Environment  |
| Squad coordinator              | src/env/env/SquadCoordinator.java                                          | Environment  |
| Dashboard (WebSocket)          | src/env/env/HiveDashboard.java                                             | Environment  |
| EIS bridge / translation       | src/env/connection/{EISAccess,Translator}.java                             | Environment  |
| Organization                   | src/org/hive_org.xml                                                       | Organization |
| JaCaMo configuration           | hive.jcm                                                                   | JaCaMo       |

### 3.5 Monitoring Dashboard — HIVE Command Center

To facilitate debugging, performance analysis, and visual demonstration of the running MAS, the **HIVE Command Center** was developed — a real-time web dashboard built with React, TypeScript, Zustand, and React Three Fiber. The dashboard receives data via WebSocket (port 8765) directly from the HiveDashboard (CArtAgO) artifact, displaying the full simulation state without impacting agent performance.

**2D View (Figure 1).** The two-dimensional interface is organized into specialized panels:

- **Agents (20)** — Grid with all 20 agents showing identifier (A1–A20), current position (x,y), destination (destX,destY), energy bar, last executed action, and result;

- **Squads** — View of the 4 squads with members, roles (leader, collector, assembler, sentinel), carried blocks, and meeting point;
- **Task Pipeline** — Visual pipeline of each task with the phases AUC → COL → MEE → CON → SUB → DON, showing assigned squad, reward, and deadline;
- **Event Feed** — Real-time event log (score updates, submissions, auctions, failures, block collection);
- **Auction Hall** — View of auctions with each squad’s bids and the winner;
- **Battle Stats** — Counters of submissions, connections, collected blocks, won auctions, failures, and alerts;
- **Score Timeline** — Chart of score evolution over the simulation steps.

The header shows connection status (LIVE/OFFLINE/SIM), current step, accumulated score, 2D/3D toggle, and an integrated fake-simulation button that demonstrates the dashboard without the MASSim server.

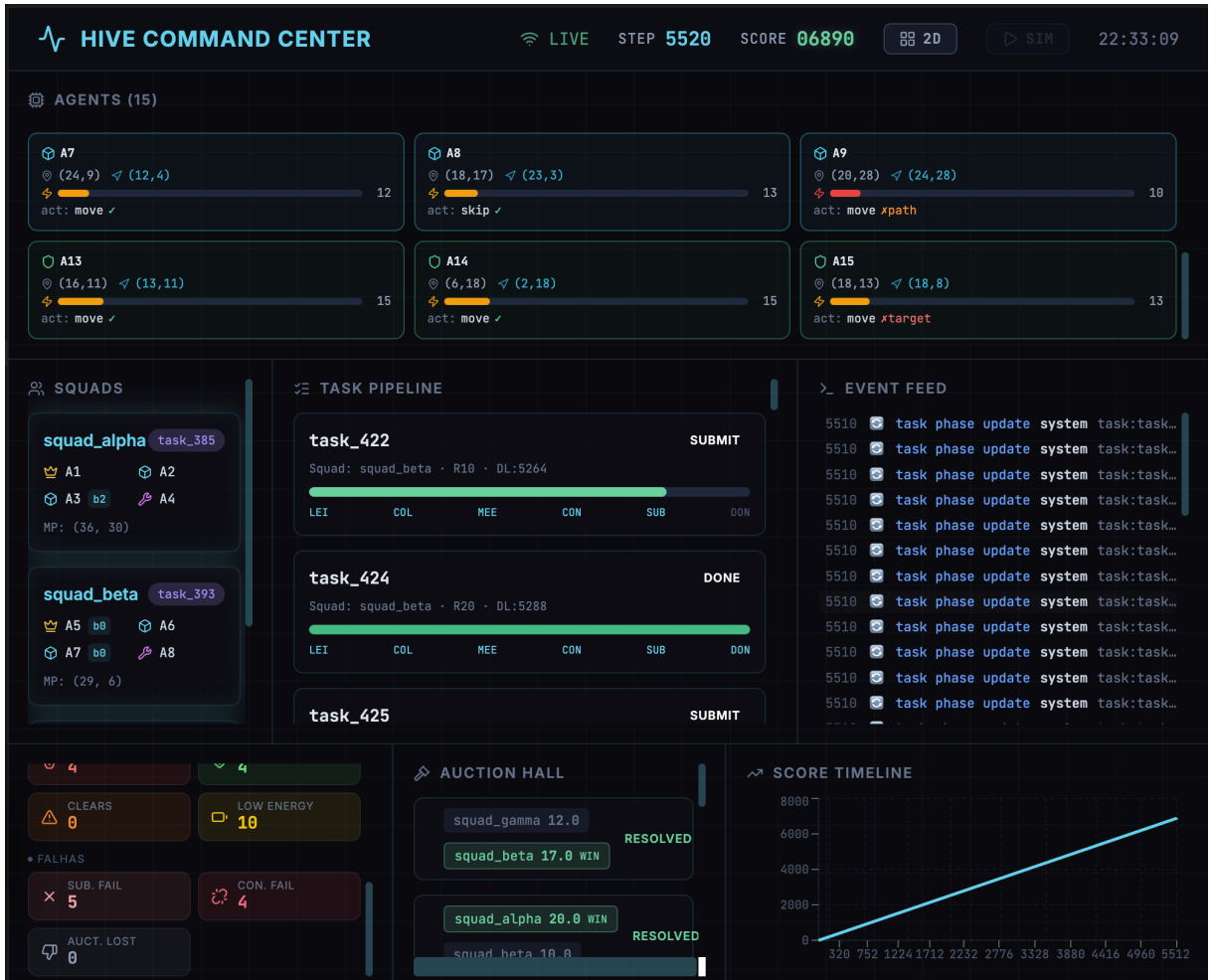


Figure 1: HIVE Command Center: 2D view with agents, squads, task pipeline, event feed, auction hall, battle stats, and score timeline.

**3D View (Figure 2).** The three-dimensional visualization renders the MASSim environment in an interactive perspective (orbit, pan, zoom) using React Three Fiber and Three.js:



- **Agents** are cubes colored by role (yellow = leader, cyan = collector, purple = assembler, green = sentinel), with a distinct size for leaders; they move via linear interpolation, and deactivated agents fall over and become translucent;
- **Dispensers** appear as rotating vertical boxes colored by block type (red = b0, blue = b1, amber = b2, purple = b3);
- **Goal zones** are semi-transparent green planes on the grid floor;
- The **HUD** shows counts of agents, dispensers, and goal zones; a **legend** at the bottom identifies the visual elements.

The side panel in the 3D view keeps the Event Feed, Battle Stats, and Score Timeline visible for simultaneous monitoring.

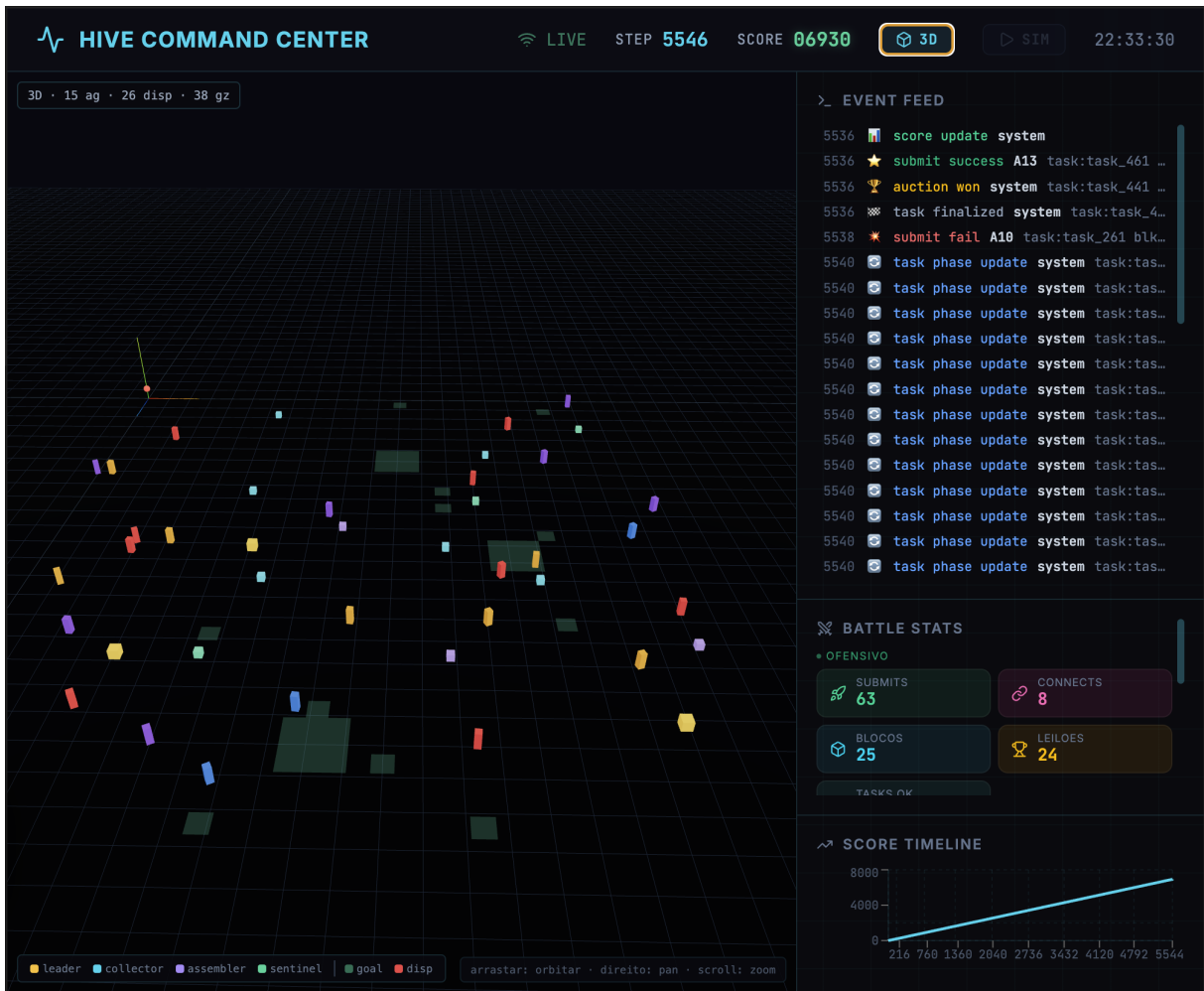


Figure 2: HIVE Command Center: 3D view with agents on the grid, rotating dispensers, goal zones, and side event panel.

The dashboard also includes an embedded **fake-simulation mode**: clicking the SIM button (when offline), the frontend internally generates simulated agents with movement, tasks, auctions, and a growing score, allowing all visual features to be demonstrated without running the MASSim server or the JaCaMo agents. When a real simulation is in progress (via WebSocket), the button is automatically disabled.

## 4 Programming Languages and Execution Platform

### 4.1 JaCaMo and the MAOP Paradigm

The **JaCaMo** platform [6, 7] implements the Multi-Agent Oriented Programming (MAOP) paradigm, which decomposes a MAS into three orthogonal dimensions:

| Dimension    | Technology   | Language      | Central abstraction                          |
|--------------|--------------|---------------|----------------------------------------------|
| Agent        | Jason [8]    | AgentSpeak(L) | Beliefs, Plans, Intentions                   |
| Environment  | CArtAgO [11] | Java          | Artifacts, Operations, Observable properties |
| Organization | MOISE+ [9]   | XML           | Roles, Groups, Deontic norms                 |

### 4.2 AgentSpeak(L) / Jason

The agents are programmed in **AgentSpeak(L)** [4, 8], a declarative language based on logic programming that translates BDI concepts into concrete constructs:

| BDI Concept    | AgentSpeak Construct                    | Example                                             |
|----------------|-----------------------------------------|-----------------------------------------------------|
| Belief         | Literal in the belief base              | <code>my_pos(10, 20)</code>                         |
| Desire / Event | Trigger event                           | <code>+!go_to(X,Y)</code>                           |
| Intention      | Running plan instance                   | <code>+!collect(B) : ... &lt;- ...</code>           |
| Plan           | <code>event : context &lt;- body</code> | <code>+!go_to(X,Y) : my_pos(X,Y) &lt;- true.</code> |

Jason’s **reasoning cycle** implements the BDI loop in 10 steps [8]: perception → BRF → event generation → event selection → relevant plans → applicable plans → plan selection → intention update → intention selection → execution. This cycle is **interpreted**, granting expressiveness but introducing overhead relative to compiled implementations.

### 4.3 CArtAgO

Artifacts are implemented as **Java classes** extending `cartago.Artifact` [11]. Each operation is annotated with `@OPERATION` and may define output parameters with `OpFeedbackParam`. Observable properties are declared via `defineObsProperty` and updated with `updateObsProperty`, generating belief events in the agents that focus the artifact.

The **per-artifact serialization** [12] ensures that all invocations on the same artifact are processed sequentially, eliminating the need for explicit locks but limiting throughput under high concurrency.

### 4.4 MOISE+

The organizational specification is declared in **XML** following the MOISE+ model [9, 10], with three documents: structural (roles, groups, relations), functional (schemes, missions, goals), and deontic (obligations and permissions binding roles to missions). The MOISE+ runtime within JaCaMo automatically enforces norm compliance by the agents.

## 4.5 Implementation Characteristics

| Characteristic                         | Value                                                     |
|----------------------------------------|-----------------------------------------------------------|
| Lines of AgentSpeak(L)                 | ~3,500 (.asl modules)                                     |
| Lines of Java (artifacts + connection) | ~2,500                                                    |
| .asl files                             | ~14 (10 common + 4 roles)                                 |
| CARTAgO artifacts                      | 4 (SharedMap, TaskBoard, SquadCoordinator, HiveDashboard) |
| MOISE+ configuration                   | 1 XML file ( <code>hive_org.xml</code> )                  |
| Build system                           | Gradle + Java 21                                          |

## 5 Strategy for the Agent Team

### 5.1 Movement Algorithm

Navigation on the 40×40 toroidal grid uses two complementary algorithms:

**A\* with toroidal wrapping:** used when the toroidal Manhattan distance to the destination is  $\leq 60$ . The heuristic is  $\min(|\Delta x|, W - |\Delta x|) + \min(|\Delta y|, H - |\Delta y|)$ , which is admissible and guarantees optimality. The expansion limit is 2,000 nodes, falling back to greedy if exceeded. Registered obstacles expire after 30 steps to accommodate environmental changes.

**Greedy navigation:** used for distances  $> 60$  or as an A\* fallback. It computes the cardinal direction that minimizes the toroidal Manhattan distance to the destination.

**Frontier-based exploration:** the SharedMap keeps a cache of **frontiers** (free cells adjacent to unvisited cells). Agents query `get_nearest_frontier` to select the next exploration destination, progressively covering the map.

### 5.2 Coordination Strategy

Coordination is carried out at three levels:

**Level 1 — Organizational (MOISE+):** the structure of 4 squads with local autonomy ensures that a leader’s failure does not impact the other squads. Each leader makes allocation decisions independently.

**Level 2 — Distributed auction (adapted Contract Net)** [22]: the task-allocation flow follows the steps below.

1. MASSim announces a new task (percept)  $\rightarrow$  TaskBoard executes `register_task`;
2. Leaders L1 and L2 submit `place_bid(score)` to the TaskBoard;
3. L1 invokes `resolve_auction`  $\rightarrow$  TaskBoard returns `winner = L1`;
4. L1 calls `find_free_soloist(dispatchX, dispatchY)` on the SquadCoordinator  $\rightarrow$  returns C3;
5. L1 sends `.send(achieve, go_collect(...))` to Collector C3;

6. C3 navigates → collects → `submit(task)` on MASSim → +10 points.

Two adaptations relative to the classic CNP: (i) **distributed manager** — any leader resolves the auction, with no central point of failure; (ii) **universal contractor** — any free agent can be a soloist, regardless of its role.

**Level 3 — Self-assignment:** idle agents self-assign an available task every 5 steps, without communication overhead.

### 5.3 Adaptation to the Official Scenario

The official MAPC 2022 configuration imposes three differences with strong impact on team strategy, absent from the development configuration:

**Reactive role adoption:** in the official setting, the initial **default** role **does not include** the actions `request`, `attach`, `connect`, and `submit` — the agent must be on a *role zone* and execute `adopt(worker)` to enable them. HIVE detects this mode **reactively and self-detectably**: upon receiving the failure code `failed_role` (which never occurs in dev, where **default** has all actions), the agent prioritizes navigation to the nearest *role zone* and adopts **worker**. Of the five official roles (**default**, **worker**, **constructor**, **explorer**, **digger**), HIVE universally adopts **worker**, since it is the only one combining all collect-assemble-submit actions and is the fastest without load (`speed [2,1,1,0]`).

**Relative positioning (dead-reckoning):** with `absolutePosition: false`, the agent perceives only relative coordinates. Each agent keeps a private local frame, integrating each successful move, and the shared map becomes a per-agent instance (`map_<name>`). Coordinates exchanged by message are *cross-frame*; map fusion requires a mutual-sighting *handshake* (future work).

**Grid dimension inference:** the official grid (70×70) is larger than the development one (40×40); A\* operates with graceful degradation when the real toroidal dimensions are not inferred at runtime.

### 5.4 Task Optimization

The iterative optimization process led to the following critical improvements:

| Optimization                                                 | Impact                                              |
|--------------------------------------------------------------|-----------------------------------------------------|
| Reduce delegation from 3→1 soloist/task                      | Score 20→80 (+300%) — eliminated agent waste        |
| Leader scan: 1 task per scan (every 10 steps)                | Score 40→100 — eliminated timeouts from heavy loops |
| Remove <code>.wait()</code> from <code>quick_delegate</code> | Latency reduction (~10ms per delegation)            |
| Toroidal distance in <code>find_free_soloist</code>          | Correct selection of the nearest soloist            |
| Task timeout 200→300 steps                                   | Agents complete tasks on hard maps                  |
| Obstacle decay every 10 steps                                | 93% reduction in decay calls                        |
| JVM heap 256MB→2GB                                           | Eliminated GC pauses                                |

## 5.5 Life Cycle of a Task (1 block)

| Stage        | Action                                                              |
|--------------|---------------------------------------------------------------------|
| Step N       | Server announces task T1                                            |
| Step N+1     | Leader: bid $\rightarrow$ auction $\rightarrow$ delegate to soloist |
| Steps N+2..k | Soloist navigates via A* to the dispenser                           |
| Step N+k     | <b>request</b> + <b>attach</b> : block collected                    |
| Steps k+1..m | Navigate with the block to the goal zone                            |
| Step N+m     | <b>submit</b> $\rightarrow$ +10 points                              |

Table 3: Life cycle of a single-block task (30–80 steps).

## 6 Technical Characteristics

### 6.1 Experimental Configuration

HIVE is designed for the **official** MAPC 2022 scenario, but the systematic experimental study was conducted in a reduced **development scenario**, which enables fast iterations and isolates variables. The table contrasts the two configurations:

| Parameter          | Development                     | Official (competition)                |
|--------------------|---------------------------------|---------------------------------------|
| Grid               | 40×40, toroidal                 | 70×70, toroidal                       |
| Map                | Cave, density 0.45              | Cave, density 0.6                     |
| Steps              | 800                             | 750                                   |
| Agents (per team)  | 6–20 (scale study)              | 20                                    |
| Simultaneous teams | 1                               | 2                                     |
| Task size          | 1–2 blocks                      | 1–4 blocks                            |
| Absolute position  | Enabled                         | <b>Disabled</b> (relative)            |
| Initial role       | <b>default</b> with all actions | <b>default</b> without collect/submit |

The performance study below refers to the **development scenario**, in which the 15-agent configuration was identified as the best balance for the 40×40 grid; the system delivered for the competition runs with 20 agents in the official scenario.

### 6.2 Performance (development scenario)

13 simulations were run with 15 agents (3L + 6C + 3A + 3S), of which 7 completed:

| Run  | Score | Submissions | Status               |
|------|-------|-------------|----------------------|
| 1    | 100   | 10          | Completed            |
| 2    | 90    | 9           | Completed            |
| 3    | 80    | 8           | Completed            |
| 4    | 80    | 8           | Completed            |
| 5    | 70    | 7           | Completed            |
| 6    | 60    | 6           | Completed            |
| 7    | 60    | 6           | Completed            |
| 8–13 | —     | —           | Crash (steps 76–230) |

**Statistics:** Mean 77.1 | Median 80 | Max 100 | Min 60 |  $\sigma = 14.6$  | Completion rate 53.8%.

### 6.3 Scalability

| Agents        | Composition              | Mean score  | Crash rate |
|---------------|--------------------------|-------------|------------|
| 6             | 1L + 3C + 1A + 1S        | 23.3        | 25%        |
| 10            | 2L + 4C + 2A + 2S        | 46.7        | 40%        |
| <b>15</b>     | <b>3L + 6C + 3A + 3S</b> | <b>77.1</b> | <b>46%</b> |
| 20 (official) | 4L + 8C + 4A + 4S        | 70.0        | 50%        |

On the reduced 40×40 grid, the score grows with the number of agents up to 15, stabilizing afterward. With 20 agents, the mean drops slightly in this development scenario (70 vs 77), suggesting saturation with at most 4 simultaneous tasks. It is worth noting that the official scenario (larger 70×70 grid, with more dispensers and goal zones) mandates 20 agents — a configuration in which the saturation observed on the small grid does not apply in the same way.

### 6.4 Stability and Failure Diagnosis

About **46% of the runs** do not complete the 800 steps. Log analysis revealed a consistent pattern:

1. **Trigger** (steps 76–230): 15 agents process percepts simultaneously, generating  $\sim 600\text{--}900$  `mark_obstacle` calls per step — all serialized in the `SharedMap` artifact [12];
2. **Escalation**: the serialized time exceeds the 8-second timeout;
3. **Collapse**: the EIS library interprets the timeout as a connection failure and starts a flood of reconnections;
4. **Terminal loop**: the disconnect/reconnect cycle becomes self-sustaining indefinitely.

The collapse pattern is a causal chain: *15 agents process percepts* ( $\sim 900$  `mark_obstacle`/step)  $\rightarrow$  *CArtAgO serialization*  $> 8s$   $\rightarrow$  *server logs “No valid action”*  $\rightarrow$  *EIS attempts to reconnect*  $\rightarrow$  *flood of reconnections*, which feeds back into the reconnection attempt, sustaining the loop indefinitely.

**Evidence:** reducing the cave density from 0.45 to 0.25 dropped the crash rate to **0%**, confirming that the volume of obstacle percepts is the determining factor. This limitation is inherent to the CArtAgO serialization model [12] — it is not a defect in the agents’ logic.

## 6.5 Failure Recovery

HIVE implements recovery mechanisms in the following scenarios:

- **Stuck agent** (*stuck detection*): if an agent stays in the same position for 50 steps, the `need_detach` mechanism is triggered — the agent detaches blocks and looks for an alternative route;
- **Goal-zone blockage**: if an agent with `pending_submit` is blocked for 6+ movement attempts, it seeks an alternative goal zone via `get_alternative_goal_zone`;
- **Task expiration**: tasks pending for more than 300 steps are abandoned and the agent returns to the soloist pool;
- **Obstacle expiration**: obstacles in the `SharedMap` expire after 30 steps, avoiding permanently invalid routes;
- **EIS timeout**: the EIS client was configured with an 8,000ms timeout, aligned with the server, to avoid premature disconnections.

## 6.6 Comparison with MAPC 2022

| Position | Team         | Framework           | Typical score |
|----------|--------------|---------------------|---------------|
| 1st      | FIT-BUT [17] | Pure Java           | 200–400       |
| 2nd      | MMD [18]     | Java (optimization) | 150–300       |
| —        | LI(A)RA [19] | JaCaMo              | 80–150        |
| —        | <b>HIVE</b>  | <b>JaCaMo</b>       | <b>60–100</b> |

The gap to the top teams is attributed to the **low completion rate of multi-block tasks**: the single-block pipeline works reliably, but the *connect* protocol rarely completes successfully on dense cave maps. Teams such as FIT-BUT [17] use pure Java with direct thread control, avoiding CArTAgo’s serialization bottleneck.

## 7 Discussion and Conclusion

### 7.1 Ease/Difficulty of the MAOP Method

The MAOP approach via JaCaMo [6] demonstrated **high conceptual expressiveness**: the separation into agents/environment/organization mapped directly onto the theoretical MAS concepts studied in the PCS 5703 course [1]. Programming in AgentSpeak(L) [8] allowed complex behaviors to be expressed declaratively and modularly.

However, Jason’s **interpreted** nature — unifying plans against the belief base every step — introduces significant overhead under high load: with 15–20 agents each running hundreds of beliefs, deliberation time becomes a limiting factor. This observation is consistent with the LTI-USP experience [20] in earlier MAPC editions.

The trade-off between **declarative expressiveness** and **computational efficiency** is the main tension found: interpreted BDI frameworks offer traceability and conceptual fidelity but lose performance to compiled Java implementations [17].

## 7.2 Ease/Difficulty of the Organizational Model

MOISE+ [9, 10] provided a clear and verifiable organizational structure that facilitated the system’s top-down design. The division into 4 squads with local autonomy demonstrated the concept of **decentralized organization**.

However, MOISE+ is primarily **declarative and static**: it defines *who can do what*, but not *when* or *how* coordination occurs at runtime. Dynamic coordination — deciding which agent takes which task, when to abandon a task — was implemented entirely in AgentSpeak plans and CArtAgO artifacts, not in the organizational model. For highly dynamic scenarios like the MAPC, MOISE+ works better as a **structural framework** than as an **operational coordination mechanism**.

## 7.3 CArtAgO: Power and Limitations

CArtAgO artifacts [11, 13] validated the central premise of the A&A meta-model [13] — artifacts as tools that mediate collective activity. The **SharedMap** encapsulated A\*, exploration, and obstacles in a clean interface; the **TaskBoard** implemented the auction without the agents knowing the protocol.

However, the **per-artifact serialization** [12] proved to be the main bottleneck. One solution would be **functional partitioning** (sharding): splitting the **SharedMap** into smaller per-function artifacts (obstacles, dispensers, goal zones), distributing the serialization load.

## 7.4 Adaptation to the Official Scenario

Migrating from the development configuration to the official scenario exposed valuable lessons. The initial **default** role enables neither collection nor submission: the agent must adopt **worker** on a *role zone*. The **reactive** implementation of this mechanism — triggered by the **failed\_role** failure code, without static mode flags — proved consistent with the BDI principle of perception-guided adaptation [3, 8] and avoided rigid assumptions about the environment. The **relative positioning** (**absolutePosition: false**), handled by dead-reckoning with a per-agent local frame, revealed the cost of giving up a global reference: map fusion among agents (a prerequisite for reliable multi-block rendezvous) requires a mutual-sighting *handshake*, left as future work. These adaptations confirm that seemingly small scenario decisions (per-role actions, coordinate reference frame) have deep architectural impact.

## 7.5 Possible Extensions

1. **Artifact sharding**: partition the **SharedMap** to reduce serialization contention;
2. **Robust multi-block pipeline**: redesign the *connect* protocol with dynamic meeting points and fault tolerance;
3. **Map fusion in relative mode**: mutual-sighting *handshake* to translate dead-reckoned frames into a common reference;
4. **Self-assignment with spatial affinity**: lightweight dispersion based on position hashing, without **findall**;
5. **Asynchronous CArtAgO operations**: non-blocking operations for map updates;
6. **Reinforcement learning for parameters**: automatic tuning of thresholds and frequencies.



## 7.6 Conclusion

This work presented HIVE, a JaCaMo-based MAS [6] for the Agents Assemble scenario of MAPC 2022 [14]. The system demonstrates the integrated application of the three MAOP pillars: BDI agents in Jason [8], normative organization in MOISE+ [9], and shared environment in CArtAgO [11].

The experimental results — scores of 60–100 points with a mean of 77 in the development scenario — validate the feasibility of the MAOP paradigm [7] for complex multi-agent coordination scenarios, while the scalability analysis (15 agents as the best balance on the reduced grid; 20 agents in the official configuration) and the instability diagnosis (CArtAgO serialization bottleneck) contribute practical insights to the community.

The main contribution goes beyond the numerical results: the experience concretely exposed the trade-offs theorized in the literature — between autonomy and coordination [1], between expressiveness and efficiency [8], between consistency and concurrency [12]. As argued by Bratman [3], rational agents need intentions as commitments that reduce the deliberation space. HIVE shows that this need applies to the **system as a whole**: the choice of framework, the granularity of artifacts, and the coordination frequency are themselves design commitments that determine the MAS's success in competitive and dynamic environments.

## References

- 1 M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.
- 2 M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
- 3 M. E. Bratman, *Intention, Plans, and Practical Reason*. Cambridge, MA: Harvard University Press, 1987.
- 4 A. S. Rao and M. P. Georgeff, “BDI Agents: From Theory to Practice,” in *Proc. 1st Int. Conf. on Multiagent Systems (ICMAS-95)*, 1995, pp. 312–319.
- 5 A. S. Rao and M. P. Georgeff, “Modeling Rational Agents within a BDI-Architecture,” in *Proc. 2nd Int. Conf. on Principles of Knowledge Representation and Reasoning (KR’91)*, 1991, pp. 473–484.
- 6 O. Boissier, R. H. Bordini, J. F. Hübner, and A. Ricci, *Multi-Agent Oriented Programming: Programming Multi-Agent Systems Using JaCaMo*. Cambridge, MA: MIT Press, 2020.
- 7 O. Boissier, R. H. Bordini, J. F. Hübner, A. Ricci, and A. Santi, “Multi-agent oriented programming with JaCaMo,” *Science of Computer Programming*, vol. 78, no. 6, pp. 747–761, 2013.
- 8 R. H. Bordini, J. F. Hübner, and M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak using Jason*. Chichester, UK: John Wiley & Sons, 2007.
- 9 J. F. Hübner, J. S. Sichman, and O. Boissier, “Developing organised multiagent systems using the MOISE+ model,” *Int. J. of Agent-Oriented Software Engineering*, vol. 1, no. 3/4, pp. 370–395, 2007.
- 10 J. F. Hübner, J. S. Sichman, and O. Boissier, “A Model for the Structural, Functional and Deontic Specification of Organizations in Multiagent Systems,” in *SBIA 2002*, LNAI, vol. 2507. Springer, 2002, pp. 118–128.
- 11 A. Ricci, M. Piunti, M. Viroli, and A. Omicini, “Environment Programming in CArtAgO,” in *Multi-Agent Programming*, Springer, 2009, pp. 259–288.
- 12 A. Ricci, M. Piunti, and M. Viroli, “Environment programming in multi-agent systems: An artifact-based perspective,” *Autonomous Agents and Multi-Agent Systems*, vol. 23, pp. 158–192, 2011.
- 13 A. Omicini, A. Ricci, and M. Viroli, “Artifacts in the A&A meta-model for multi-agent systems,” *Autonomous Agents and Multi-Agent Systems*, vol. 17, no. 3, pp. 432–456, 2008.

- 14 T. Ahlbrecht, J. Dix, N. Fiekas, and T. Krausburg, Eds., *The Multi-Agent Programming Contest 2022*, LNCS, vol. 13997. Springer, 2023.
- 15 T. Ahlbrecht, J. Dix, N. Fiekas, and T. Krausburg, Eds., *The Multi-Agent Programming Contest 2021*, LNCS, vol. 12947. Springer, 2021.
- 16 T. Ahlbrecht, J. Dix, N. Fiekas, and T. Krausburg, “The Multi-Agent Programming Contest: A Résumé,” *Annals of Mathematics and Artificial Intelligence*, 2020.
- 17 F. Zboril Jr., F. Vidensky, L. Dokoupil, and J. Beran, “FIT-BUT at MAPC 2022,” in *LNCS*, vol. 13997. Springer, 2023, pp. 120–150.
- 18 P. Böhm, “Optimization-Based Agents in the 16th Multi-agent Programming Contest,” in *LNCS*, vol. 13997. Springer, 2023, pp. 19–53.
- 19 M. Custódio, M. Rocha, R. Battaglin, G. P. Farias, and A. R. Panisson, “LI(A)RA Team — MAPC 2022,” in *LNCS*, vol. 13997. Springer, 2023, pp. 165–194.
- 20 M. F. Stabile Jr. and J. S. Sichman, “The LTI-USP Strategy to the 2020/2021 Multi-Agent Programming Contest,” in *LNCS*, vol. 12947. Springer, 2021, pp. 108–133.
- 21 M. R. Franco, L. M. Rosset, and J. S. Sichman, “Improving the LTI-USP Team: A New JaCaMo Based MAS for the MAPC 2013,” in *EMAS 2013*, LNAI, vol. 8245. Springer, 2013, pp. 339–348.
- 22 R. G. Smith, “The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver,” *IEEE Trans. on Computers*, vol. C-29, no. 12, pp. 1104–1113, 1980.
- 23 Multi-Agent Programming Contest, “Scenario Description: Agents Assemble 2022.” [Online]. [https://github.com/agentcontest/massim\\_2022/blob/main/docs/scenario.md](https://github.com/agentcontest/massim_2022/blob/main/docs/scenario.md)
- 24 Multi-Agent Programming Contest. [Online]. <https://multiagentcontest.org/>
- 25 JaCaMo Project. [Online]. <https://jacamo-lang.github.io/>